

Programación II

Desarrollo en Java

Clase N° 18

Manejo de archivos JSON

Profesora Carolina Archuby



¿QUÉ ES JSON ?

=> JSON (acrónimo de Java Script Object Notation) es un FORMATO LIGERO DE INTERCAMBIO DE DATOS que se utiliza para **TRANSMITIR INFORMACIÓN ESTRUCTURADA ENTRE DIFERENTES APLICACIONES.**

=> Se basa en una sintaxis de objetos y arrays, representada en TEXTO, que **permite representar datos simples y complejos** de forma sencilla, liviana, e INDEPENDIENTE DEL LENGUAJE DE PROGRAMACION.

=> NO es un lenguaje de programación, es un formato de intercambio de datos.

CARACTERÍSTICAS Y VENTAJAS DE USAR JSON

- => **Ligereza**: Es un formato de datos muy ligero, lo que lo hace ideal para aplicaciones web y móviles donde el ancho de banda es un recurso valioso.
- => **Fácil de leer y escribir**: Utiliza una sintaxis simple basada en texto, fácil de leer y escribir tanto para humanos como para máquinas.
- => **Independiente del lenguaje**: Es compatible con muchos lenguajes de programación y plataformas, lo que lo hace ideal para aplicaciones que necesitan comunicarse con diferentes sistemas.
- => **Flexibilidad**: Admite estructuras de datos simples y complejas, como arrays y objetos anidados, lo que lo convierte en un formato muy flexible y escalable.
- => **Gratis, libre y documentado**

PARA QUÉ SE USA JSON

- => Es una herramienta fundamental para el intercambio de datos en Java.
- => Intercambio de datos entre front-end y back-end (APIs REST).
- => Almacenamiento ligero.
- => Archivos de configuración.
- => Comunicación entre servicios y microservicios.
- => Serialización y exportación de datos.
- => Permite que sistemas escritos en distintos lenguajes intercambien datos fácilmente
- => Es el estándar más usado en las APIs modernas.

JSON vs BASE DE DATOS

- => JSON complementa, no reemplaza a una Base de Datos.
- => Ventajas de JSON frente a una DB: Ligera y portable (DB es pesada). Legible para humanos (DB requiere consultas)
- => Ventajas de la DB frente a JSON: Es más potente, ofrece integridad, índices y consultas complejas y eficientes.
- => **Conclusión: JSON para transferencia y almacenamiento ligero, DB para persistencia compleja.DB**

LIBRERÍAS para utilizar JSON en Java

Para usar JSON en Java hay que usar librerías, por ejemplo:

- Jackson
- GSON
- **JSON.org**

Proporcionan métodos y mapeadores para realizar la serialización y deserialización:



SERIALIZACION => Transformar el Objeto Java a un JSONObject o JSONArray



DESERIALIZACION => Transformar el JSONObject o JSONArray a un Objeto Java

ESTRUCTURAS UTILIZADAS POR JSON

JSON trabaja con dos estructuras:

=> **JSONObject**

=> **JSONArray**

JSONObject - {"clave": "valor" }

=> Colección no ordenada de **pares key (nombre campo) / value (valor campo)**.

=> Los valores pueden ser tipos de **datos nativos de Java** (cadenas, números, booleanos, nulos), y también **objetos JSON** y **arreglos JSON**.

La clase en Java:

```
class Persona {  
    String nombre;  
    int edad;  
    String ciudad;;  
}
```

Su representación en JSONObject:

```
{  
    "nombre": "Juan",  
    "edad": 30,  
    "ciudad": "Buenos Aires"  
}
```


JSONObject

=> Cuenta los métodos **PUT** y **GET**.

=> **PUT**: * Sobrecargado para todo tipo de datos (acepta tipos de datos del lenguaje Java, objetos JSON y arreglos JSON)

- * Usa como argumentos una **clave o nombre (String)** y un **valor**.
- * Arroja **JSONException**, por lo que debe invocarse dentro de un bloque **try- catch**.

=> **GET**: * Se usa **GET+TipoDeDato** para **obtener el valor de una clave** enviada como parámetro).

JSONArray - [elemento1, elemento2, elemento3]

- => Secuencia ordenada de **ceros o más JSONObject** .
- => Los datos no se registran con clave-valor, se usa un **índice**.
- => Los elementos dentro del arreglo se separan por coma (de manera automática)
- => Cuenta con método **LENGTH** para conocer la longitud del mismo.
- => Cuenta con un método **PUT** para **ingresar elementos** al arreglo.
- => Cuenta con un método **GET** para **acceder a un elemento** en particular
(arreglo.getJSONObject(i)).

JSONArray – Su representación:

```
[  
  {  
    "nombre": "Juan",  
    "apellido": "Pérez",  
    "edad": 30  
  },  
  {  
    "nombre": "María",  
    "apellido": "Gómez",  
    "edad": 25  
  },  
  {  
    "nombre": "Carlos",  
    "apellido": "Rodríguez",  
    "edad": 35  
  }  
]
```

Un ejemplo de estructura anidada:

Un objeto que contiene dentro:

* un arreglo JSON de objetos

(notar que se usan corchetes, representativos de los JSONArray)

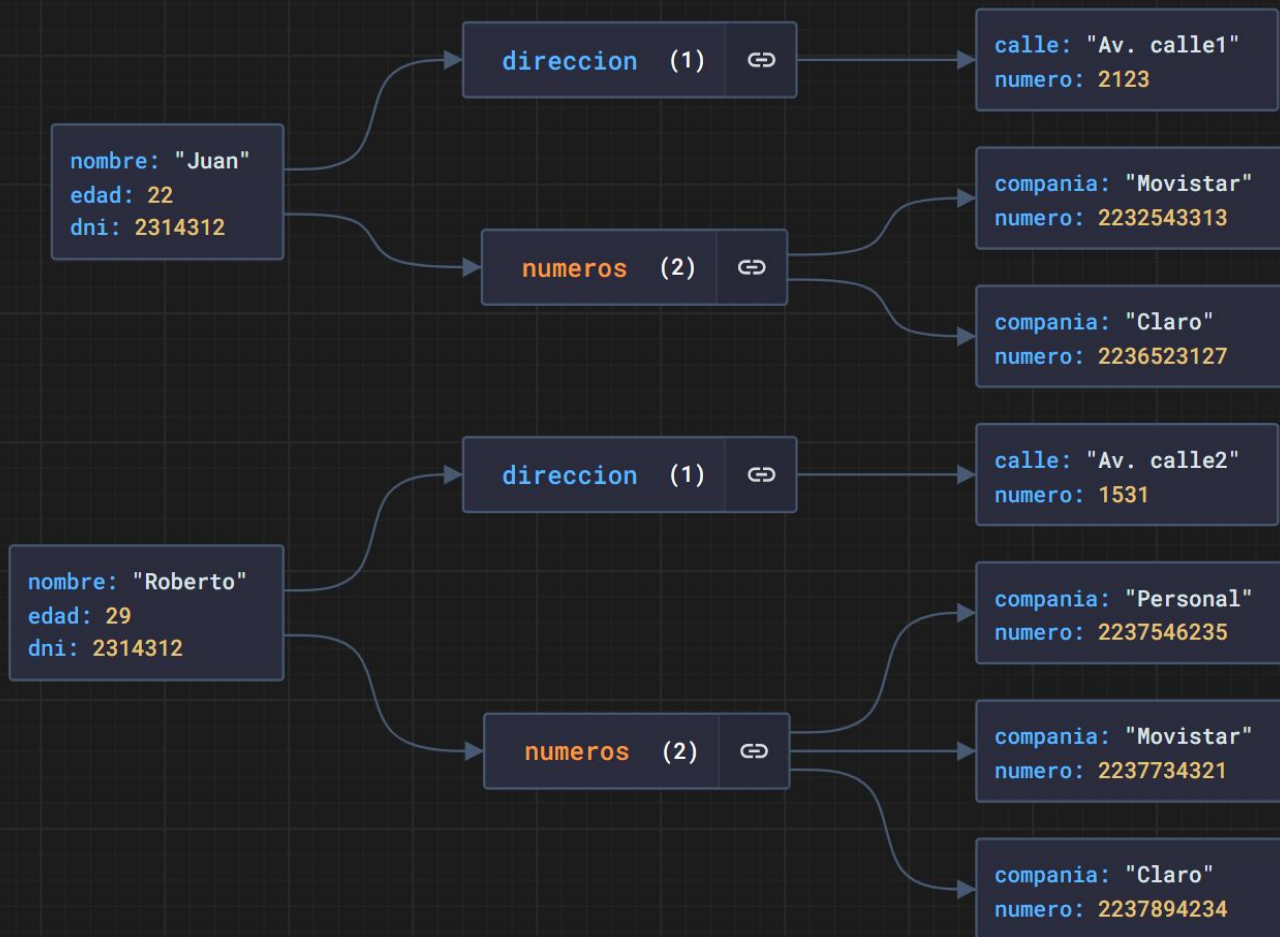
* otro objeto JSON

(notar que se usan llaves, representativas de los JSONObject)

```
{  
  "nombre": "Juan",  
  "apellido": "Pérez",  
  "edad": 30,  
  "ciudad": "Buenos Aires",  
  "telefonos": [  
    "123456789",  
    "987654321"  
  ],  
  "direccion": {  
    "calle": "Calle Principal",  
    "numero": 123,  
    "ciudad": "Buenos Aires"  
  }  
}
```

Otro ejemplo de estructuras anidadas:

```
1  {
2    "nombre": "Roberto",
3    "edad": 29,
4    "dni": 2314312,
5    "direccion": {
6      "calle": "Av. calle2",
7      "numero": 1531
8    },
9  },
10  "numeros": [
11    {
12      "compania": "Personal",
13      "numero": 2237546235
14    },
15    {
16      "compania": "Movistar",
17      "numero": 2237734321
18    },
19    {
20      "compania": "Claro",
21      "numero": 2237894234
22    }
23  ]
24  },
25  {
26    "nombre": "Juan",
27    "edad": 22,
28    "dni": 2314312,
29    "direccion": {
30      "calle": "Av. calle1",
31      "numero": 2123
32    },
33    "numeros": [
34      {
35        "compania": "Movistar",
36        "numero": 2232543313
37      },
38      {
39        "compania": "Claro",
40        "numero": 2236523127
41      }
42    ]
43  }
44  ]
```





SERIALIZACION EN LA LIBRERÍA ORG.JSON

=> SERIALIZAR: transformar el Objeto Java a un JSONObject o JSONArray

=> PASOS: Creamos clases gestoras que nos ayuden:

- * Una clase “**GestoraJSON**” que se ocupe de las operaciones de Serialización y Deserialización.

- * Una clase “**OperacionesLectoEscritura**” que se ocupe de las operaciones de grabado y lectura de información del archivo.

1) METODO PRINCIPAL dentro de la clase “GestoraJSON”:

- * Método que se invoca desde el Main
- * Se ocupa de invocar al que serializa y luego al que graba en el archivo.

```
public void listaToArchivo(ArrayList<Persona> lista, String nombreArchivo) {  
    OperacionesLectoEscritura.grabar(nombreArchivo, serializarLista(lista));  
}
```

2) Métodos para SERIALIZAR dentro de la clase "GestoraJSON":

Ejemplo:
Una lista
de
personas:

```
public class Persona {  
    private String nombre;  
    private int edad;  
    private String dni;  
    private String sexo;  
}
```

```
public JSONObject serializarPersona(Persona p){  
    JSONObject jsonObject = null;  
    try{  
        jsonObject = new JSONObject();  
        jsonObject.put("nombre", p.getNombre());  
        jsonObject.put("edad", p.getEdad());  
        jsonObject.put("sexo", p.getSexo());  
        jsonObject.put("dni", p.getDni());  
    } catch (JSONException e) {  
        e.printStackTrace();  
    }  
    return jsonObject;  
}
```

```
public JSONArray serializarLista(ArrayList<Persona>lp){  
    JSONArray jsonArray = null;  
    try{  
        jsonArray = new JSONArray();  
        for (Persona p: lp){  
            // serializamos cada persona  
            // y agregamos su JSONObject al JSONArray  
            jsonArray.put(serializarPersona(p));  
        }  
    } catch (JSONException e) {  
        e.printStackTrace();  
    }  
    return jsonArray;  
}
```


3) Método estático para GRABAR EL ARCHIVO dentro de la clase "OperacionesLectoEscritura":

```
public static void grabar(String nombreArchivo, JSONArray jsonArray) {  
    try {  
        FileWriter fileWriter = new FileWriter(nombreArchivo);  
        fileWriter.write(jsonArray.toString(indentFactor: 4));  
        fileWriter.close();  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
}
```



DESERIALIZACION EN ORG.JSON

=> DESERIALIZAR: transformar el JSONObject o JSONArray a un Objeto Java

=> Usamos las mismas **clases gestoras** de antes:
“GestoraJSON” y “OperacionesLectoEscritura”

1) METODO PRINCIPAL dentro de la clase “GestoraJSON”:

- * Método que se invoca desde el Main
- * Se ocupa de invocar al que lee el archivo y luego al que deserializa el JSON

```
public ArrayList<Persona> archivoAlista(String nombreArchivo){  
    JSONTokener tokenizer= OperacionesLectoEscritura.leer(nombreArchivo);  
    ArrayList<Persona> lista= null;  
    try{  
        lista = deserializarLista(new JSONArray(tokenizer));  
    } catch (JSONException e) {  
        e.printStackTrace();  
    }  
    return lista;  
}
```

2) Método estático para LEER EL ARCHIVO dentro de la clase "OperacionesLectoEscritura":

```
public static JSONTokener leer(String nombreArchivo) {  
    JSONTokener tokener = null;  
    try {  
        tokener = new JSONTokener(new FileReader(nombreArchivo));  
    } catch (FileNotFoundException e) {  
        e.printStackTrace();  
    } catch (JSONException e) {  
        e.printStackTrace();  
    }  
    return tokener;  
}
```

2) Métodos para DESERIALIZAR dentro de la clase "GestoraJSON":

```
public ArrayList<Persona> deserializarLista (JSONArray jArray) {  
    ArrayList<Persona> lista = new ArrayList<>();  
    try{  
        //cada Persona es un JSONObject dentro del JSONArray de Personas  
        //deserializamos 1x1 en el for y los agregamos a a lista de Personas  
        for (int i=0; i< jArray.length(); i++){  
            Persona p= deserializarPersona(jArray.getJSONObject(i));  
            lista.add(p);  
        }  
    } catch (JSONException e) {  
        e.printStackTrace();  
    }  
    return lista;  
}
```



```
public Persona deserializarPersona(JSONObject jobject){  
    Persona p= new Persona();  
    try{  
        p.setNombre(jobject.getString( key: "nombre"));  
        p.setDni(jobject.getString( key: "dni"));  
        p.setEdad(jobject.getInt( key: "edad"));  
        p.setSexo(jobject.getString( key: "sexo"));  
    } catch (JSONException e) {  
        e.printStackTrace();  
    }  
    return p;  
}
```

CONSIDERACIÓN:

Siempre debemos conocer la ESTRUCTURA
de nuestro JSON

(no el contenido, porque será variable, obvio)

Tanto en archivos como webservice



ERRORES COMUNES A EVITAR AL UTILIZAR JSON

- Asegurarse de que la **sintaxis** del **archivo JSON** sea **válida**: el archivo debe tener un formato de objeto JSON válido y estar bien estructurado.
- Verificar que los **valores** en el archivo JSON sean del **tipo correcto** (cadena, número, booleano, etc) según lo que se espera en tu aplicación.
- Asegurarse de que los **nombres de las propiedades** estén escritos correctamente y sean coherentes en todo el archivo JSON.
- Asegurarse de que las propiedades privadas tengan getters/setters.

BUENAS PRÁCTICAS AL UTILIZAR JSON

- **Respetar estándares** de codificación y convenciones de nomenclatura
- Utilizar **nombres de propiedades descriptivos y consistentes** en todo el archivo JSON, preferiblemente en minúsculas y separados por guiones bajos para facilitar la lectura.
- Utilizar **indentación** para estructurar el archivo JSON, lo que lo hace más fácil de leer y de mantener.
- Usa **comillas dobles** para los nombres de las propiedades y para los valores de cadena, ya que es el estándar aceptado en la mayoría de las bibliotecas JSON.



CONSIDERACIONES DE SEGURIDAD AL USAR JSON

- **Validar** el archivo JSON antes de analizarlo, para detectar y prevenir cualquier contenido malintencionado, ya que un archivo JSON mal formado puede ser utilizado para ejecutar ataques de inyección de código.
- **No confiar** en la fuente del archivo JSON, ya que los datos pueden ser manipulados o alterados antes de ser guardados en el archivo.
- Si estás trabajando con datos **confidenciales**, asegúrate de **encriptar** el archivo JSON y de protegerlo con medidas de seguridad adicionales, como **autenticación** y **autorización**.

Links útiles:

- Visualizadores/validadores:
 - <http://jsonviewer.stack.hu/>
 - <https://jsonlint.com>
- Generador random: <http://www.json-generator.com/>
- Editores:
 - <http://tomeko.net/software/JSONedit/index.php?lang=en>
 - <http://www.jsoneditoronline.org/>
- Comparación: <http://blog.takipi.com/the-ultimate-json-library-json-simple-vs-gson-vs-jackson-vs-json/>